

SUPERMARKET MANAGEMENT SYSTEM

Project Documentation

A Complete Guide to the Project's Functionality and Design

Python | Object-Oriented Programming (OOP) | Tkinter GUI | Data Visualization

1. Project Overview

This project is a Supermarket Management System built in Python, running as a desktop GUI application using the tkinter library, that simulates a real shopping experience inside a supermarket. It allows the user to browse and search available products by category or keyword, sort the product list by name, price, stock, or discount, add products to a shopping cart, fully manage the cart contents (update quantity, remove a product, or clear the cart entirely), generate and save a receipt containing the purchase details and totals, and view visual statistics about sales and stock through charts.

The tkinter and ttk libraries are used to build the graphical interface (windows, tables, buttons, dialogs), the datetime library is used to automatically generate the receipt number, date, and time, and the matplotlib library is used to draw the statistics charts (pie chart and bar charts).

1.1 Project Goals

- A practical application of Object-Oriented Programming (OOP) concepts in a real, graphical project rather than just theoretical examples.
- Simulating a complete purchase lifecycle: browsing/searching products → adding to cart → managing the cart → generating and saving the receipt.
- Practicing teamwork, since the project was divided among 5 members, each responsible for an independent module of the system.
- Implementing inventory management, where stock is reserved when items are added to the cart and restored when removed, reduced, or the cart is cleared.
- Providing business insight through a statistics dashboard that visualizes products per category, current stock levels, quantities sold, and revenue per product.

2. System Functionality

The system is built around a main home window with a search bar, a category selector, and four primary actions (View Products, Shopping Cart, Statistics, Receipt), plus an Exit option.

2.1 Home Screen

When the program starts, the main window opens directly showing:

- A search bar that allows searching for a product by ID or by name/category keyword.
- A grid of category buttons (Fruits, Vegetables, Dairy, Drinks, Snacks, Bakery, Frozen Food, Grocery) that filter the product list when clicked.
- Main navigation buttons: View All Products, Shopping Cart, Statistics, and Receipt.
- An Exit button that closes the application.

2.2 Product Browsing, Search and Sorting

Selecting “View All Products”, a category, or performing a search displays a table (built with a ttk Treeview) listing product ID, category, name, price, stock with its unit, and discount percentage. From this table the user can select a product and add it to the cart, or go back to the home screen.

The search feature accepts either a numeric product ID for an exact match, or a text keyword that is matched against the product name and category (case-insensitive).

Every product table (whether showing all products, a single category, or a search result) also includes a "Sort By" dropdown above the table. Selecting an option immediately re-sorts and redraws the currently displayed products without losing the active category or search filter. The Sort option is applied to the current product list and resets to Default when returning to the Home Screen.

The available sort options are:

- Default: the products are shown in their original, unsorted order.
- Name (A → Z) / Name (Z → A): alphabetical order by product name, ascending or descending.
- Price (Low → High) / Price (High → Low): numerical order by unit price.
- Stock (Low → High) / Stock (High → Low): numerical order by the currently available stock quantity.
- Discount (Low → High) / Discount (High → Low): numerical order by the discount percentage.

2.3 Add To Cart

The user selects a product row in the table and clicks "Add To Cart". A dialog box asks for the desired quantity (a positive integer). The system then validates the request:

- The quantity must be provided and must be a valid positive integer (minvalue = 1 in the input dialog).
- The requested quantity must not exceed the product's currently available stock.

If the product already exists in the cart, its quantity is increased instead of creating a duplicate entry. In all cases, the added quantity is immediately deducted (reserved) from the product's stock, and the product table refreshes to reflect the updated stock.

2.4 Shopping Cart

The Shopping Cart screen displays a table of every item currently in the cart (product, quantity, unit price, discount percentage, and final price after discount), along with the cart's grand total. From here the user can:

- Update Quantity: enter a new quantity for the selected item; the system computes the true available amount (current stock + quantity already reserved in the cart) so the value can be safely increased or decreased, and rejects the change if it exceeds what is available.
- Remove Product: removes the selected item from the cart and returns its reserved quantity back to stock.
- Clear Cart: empties the cart after a confirmation prompt, returning every reserved quantity back to stock.
- Back: returns to the home screen.

If the cart is empty, the relevant screens (Shopping Cart, Receipt) inform the user with a message box instead of showing an empty table.

2.5 Receipt

The Receipt screen automatically generates a receipt number (a timestamp), the current date, and the current time. It displays a table of every purchased item with quantity, price, discount, and final price, followed by the subtotal, total discount amount, and final total. From this screen the user can:

- Save Receipt: writes a formatted, itemized copy of the receipt to a text file named Receipt_<timestamp>.txt in the working directory, and confirms the save with a message box.
- Home (Finish Order): confirms the sale, updates each purchased product's cumulative sold quantity and revenue, clears the cart, shows a thank-you message, and returns to the home screen.

2.6 Statistics

The Statistics screen provides a simple sales and inventory dashboard, rendered with matplotlib, offering four chart options:

- Products Per Category: a pie chart showing the proportion of the catalog occupied by each category.
- Stock Levels: a bar chart of the current stock quantity for every product.

- Products Sold: a bar chart of the cumulative quantity sold for every product (updated only after an order is finished).
- Revenue: a bar chart of the cumulative revenue generated by every product (updated only after an order is finished).

2.7 Exit

The user can close the application at any time from the Exit button on the home screen, or continue shopping after completing and saving a receipt, since finishing an order returns to the home screen rather than closing the program.

3. System Architecture

The entire project is built using Object-Oriented Programming (OOP). The system is divided into 4 main classes, each with a single, clear, well-defined responsibility (the Single Responsibility Principle).

3.1 Product Class — Representing a Product

This class represents the core model of a product in the supermarket. Every product is an independent object carrying its own data, including running sales statistics:

Attribute	Description
id	A unique identifier for each product
category	The product's category (Fruits, Vegetables, Dairy, etc.)
name	The product's name
price	The price per unit
stock	The quantity currently available (reserved amounts are deducted immediately)
unit	The unit of measurement (kg, pcs, box, etc.)
discount	The discount percentage applied to the product (%)
sold	The cumulative quantity sold, updated only when an order is finished
revenue	The cumulative revenue generated, updated only when an order is finished

3.2 CartItem Class — An Item Inside the Cart

This class represents a single product that has been added to the cart along with the requested quantity, linking a Product object with a quantity. It contains three methods responsible for the financial calculations of that specific item only:

- `total_price()`: The total price before discount = price × quantity.
- `discount()`: The discount value = total price × discount percentage ÷ 100.
- `final_price()`: The final price after subtracting the discount amount.

Keeping these calculations inside `CartItem`, instead of scattering them elsewhere, keeps the code organized — each cart item knows how to calculate its own price without repeating the logic in the cart view, the receipt, or the saved receipt file.

3.3 Cart Class — The Shopping Cart

Represents the shopping cart as a whole, holding a list of `CartItem` objects. Its responsibilities are:

- `clear()`: Empties the cart completely.
- `is_empty()`: Checks whether the cart is empty.

- `total()`: Calculates the cart's grand total by summing `final_price()` across all its items.

3.4 SuperMarketGUI Class — The Main Controller

This is the core class that manages the entire application. It builds and owns the tkinter root window, holds the list of available products, the list of categories, and a Cart object, and contains all the functions that handle user interaction and screen navigation, including:

- `home_screen()`: Builds the main window (search bar, categories, navigation buttons).
- `show_products()` / `show_all_products()` / `search_product()`: Display and filter the product catalog.
- `sort_products()`, `on_sort_selected()`, `populate_tree()`: Apply the currently selected sort option to a product list and redraw the Treeview table accordingly.
- `find_product()`: Looks up a specific product by its ID.
- `add_to_cart()`, `update_quantity()`, `remove_product()`, `clear_cart()`: Manage the cart's contents and keep stock reservations consistent.
- `view_cart()`: Displays and manages the cart screen.
- `show_receipt()`, `save_receipt()`, `finish_order()`: Generate the receipt, persist it to a text file, and settle the sale (updating sold/revenue and clearing the cart).
- `statistics_menu()`, `category_chart()`, `stock_chart()`, `sold_chart()`, `revenue_chart()`: Build and render the statistics dashboard using matplotlib.
- `clear_window()`: A helper that removes all widgets from the window before rendering a new screen, avoiding overlapping UI elements.

4. Why Object-Oriented Programming?

Choosing OOP for this project was not arbitrary — it was the most suitable approach for several reasons tied directly to the nature of the problem:

4.1 Real-World Modeling

A supermarket system, in reality, consists of separate, real entities: products, a shopping cart, items inside the cart, and a store application that manages everything. OOP allows each of these entities to be represented as an independent class with its own data and behavior, just as it exists in real life, which makes the code much easier to read and understand for anyone reviewing it.

4.2 Single Responsibility Principle

Every class in the project has exactly one responsibility: Product only represents product data and sales statistics, CartItem only calculates the details of a single cart entry, Cart manages the list of items as a whole, and SuperMarketGUI manages the interface and the user's interaction with the system. This separation means that a future change in one part is unlikely to affect the rest of the system.

4.3 Reusability

Instead of repeating the logic for calculating the final price or discount everywhere it's needed, it was written once inside the CartItem methods and reused wherever required (in the cart view, the receipt screen, and the saved receipt file). The same applies to helper functions like `clear_window()` and `find_product()`.

4.4 Easier Team Collaboration

Since the project was divided among 5 team members, splitting the code into classes and clearly separated functions made it easy for each member to work on an independent part of the system without conflicting with their teammates' work — this is clearly visible from the code comments dividing the file into Member 1 through Member 5.

4.5 Easier Future Scalability

If the team wanted to add new features in the future — such as a customer login system, multiple payment methods, or persistent storage in a database — the current OOP-based structure allows these features to be added as new attributes or methods inside the existing classes, or even as entirely new classes (such as Customer or Payment), without needing to rewrite the system from scratch.

4.6 Encapsulation

Every object in the system carries its own data alongside the functions that operate on that data. For example, a CartItem object itself knows how to calculate its own final_price, and a Product object itself tracks its own sold quantity and revenue, rather than relying on separate functions that manipulate external data arbitrarily — this reduces errors and keeps the code safer and better organized.

5. Class Relationships

The relationship between the four classes can be summarized as follows:

- SuperMarketGUI
- holds a list of (1..*) → Product
- holds one (1) → Cart
- Cart holds a list of (0..*) → CartItem
- CartItem references (1) → Product

In other words, SuperMarketGUI is the main entry point that ties everything together, including the window and all its screens. Each CartItem does not store a separate copy of the product's data — it keeps a reference to the same original Product object. This guarantees that any change to the product's data (such as an updated stock level, sold count, or revenue) is immediately reflected everywhere that product is used, including the statistics charts.

6. Program Flow

1. The program starts and a SuperMarketGUI object is created, which builds the main tkinter window, initializes the product catalog, the category list, and an empty Cart, then displays the home screen.
2. From the home screen, the user searches or filters by category to browse products, then may pick a "Sort By" option (name, price, stock, or discount) to reorder the currently displayed table via sort_products(), or opens the Shopping Cart, Statistics, or Receipt screens directly.
3. Selecting a product and clicking “Add To Cart” calls add_to_cart(), which validates the quantity, reserves the stock, and updates the cart.
4. From the Shopping Cart screen, the user may call update_quantity(), remove_product(), or clear_cart() to manage the cart before checking out.
5. Opening the Receipt screen calls show_receipt(); the user may then call save_receipt() to export the receipt to a text file, and finish_order() to confirm the sale, update statistics, and clear the cart.
6. At any point the user may open statistics_menu() to view the category, stock, sold, or revenue charts, generated live from the current product data.
7. The application remains open, looping back to the home screen after each operation, until the user clicks Exit.

7. Technologies Used

Tool	Purpose
Python 3	The core programming language used to build the entire project
tkinter / ttk	Builds the graphical user interface: windows, tables (Treeview), buttons, and input dialogs

Tool	Purpose
datetime	Automatically generates the receipt number, date, and time when a receipt is issued or saved
matplotlib	Renders the statistics dashboard: pie chart and bar charts for category share, stock, sales, and revenue

8. Functional Requirements

Functional requirements describe the specific behaviors and features the system must provide to the user.

8.1 Product Management

- FR-1: The system shall display a catalog of all available products, including ID, category, name, price, stock quantity, unit, and discount.
- FR-2: The system shall allow searching for a product by its unique ID or by a keyword matched against its name or category.
- FR-3: The system shall allow filtering the product catalog by category.
- FR-4: The system shall allow sorting the currently displayed product list by name, price, stock, or discount, in ascending or descending order, while preserving the active category or search filter.

8.2 Shopping Cart Management

- FR-5: The system shall allow the user to add a selected product to the cart by specifying a quantity.
- FR-6: The system shall merge a repeated add-to-cart request for the same product into the existing cart entry rather than creating a duplicate line.
- FR-7: The system shall allow the user to view all items currently in the cart, along with quantity, unit price, discount, final price per item, and the cart's grand total.
- FR-8: The system shall allow the user to update the quantity of a product already in the cart.
- FR-9: The system shall allow the user to remove an individual product from the cart.
- FR-10: The system shall allow the user to clear the entire cart in a single action, after confirmation.

8.3 Inventory Management

- FR-11: The system shall automatically deduct (reserve) the requested quantity from a product's stock when it is added to the cart.
- FR-12: The system shall automatically restore stock when a cart item is removed, its quantity is reduced, or the cart is cleared.

8.4 Checkout, Receipt and Statistics

- FR-13: The system shall calculate the subtotal, total discount amount, and final total for the items in the cart.
- FR-14: The system shall generate a receipt containing a unique receipt number, the current date, and the current time.
- FR-15: The system shall allow the user to save the receipt to a text file.
- FR-16: The system shall allow the user to finish an order, which updates each purchased product's cumulative sold quantity and revenue, and clears the cart.
- FR-17: The system shall prevent access to the Shopping Cart and Receipt screens when the cart is empty and shall inform the user accordingly.
- FR-18: The system shall display statistics charts for products per category, current stock levels, quantities sold, and revenue per product.

8.5 Navigation

- FR-19: The system shall present a home screen with access to product browsing, the shopping cart, statistics, and the receipt.
- FR-20: The system shall allow the user to return to the home screen from every other screen.
- FR-21: The system shall allow the user to exit the application from the home screen at any time.

9. Non-Functional Requirements

Non-functional requirements describe the quality attributes and constraints under which the system must operate.

Category	Requirement
Usability	The graphical interface shall use clear labels, color-coded buttons, and icons/emoji to make screens and actions easy to recognize and navigate.
Performance	The system shall respond to user actions and refresh screens instantly, since all product and cart data is held in memory with no external I/O other than saving the receipt file.
Reliability	The system shall not crash on invalid input; quantity dialogs shall be guarded with validation and the user shall be warned via message boxes rather than the application failing.
Maintainability	The system shall follow Object-Oriented Programming principles (single responsibility, encapsulation) so that individual classes can be modified or extended independently.
Portability	The system shall run on any platform with Python 3 and the tkinter and matplotlib libraries installed.
Data Consistency	Stock levels shall remain consistent at all times: any quantity reserved when added to the cart shall be fully restored if removed, reduced, or the cart is cleared.
Scalability	The class-based architecture shall allow new features (e.g., customer accounts, payment methods, persistent/database storage) to be added without rewriting existing classes.
Usability of Output	Product, cart, and receipt data shall be displayed in aligned tables (Treeview) for readability, and statistics shall be presented as clearly labeled charts.
Availability	The system shall remain available for continuous use in a single session, returning to the home screen after every operation until the user chooses to exit.

10. Error Handling

The system anticipates invalid or unexpected user input and application state at every stage of interaction and handles it gracefully, without crashing, by validating input and displaying a clear message box before returning control to the user.

10.1 Input Validation Errors

Scenario	Handling
No product selected before adding to cart or managing a cart item	A warning message box (“Please Select A Product.”) is shown and the action is cancelled.

Scenario	Handling
Quantity dialog left empty or cancelled	The function returns immediately without modifying the product or the cart.
Quantity requested exceeds available stock	An error message box ("Not Enough Stock.") is shown and the operation is cancelled, checked against stock (or stock + quantity already in cart when updating).
Empty search keyword	A warning message box asks the user to enter a product name or ID before searching.
Search finds no matching product	An error message box ("No Product Found.") is shown.

10.2 State-Based Errors

Scenario	Handling
Opening the Shopping Cart while it is empty	An informational message box ("Shopping Cart Is Empty.") is shown and the user is returned to the home screen.
Opening the Receipt while the cart is empty	A warning message box ("Shopping Cart Is Empty.") is shown and receipt generation is blocked.
Clearing an already-empty cart	A warning message box informs the user instead of performing the action.
Clearing the cart	The user must confirm the action in a Yes/No dialog before the cart is actually cleared.

10.3 Design Principles Behind Error Handling

- Fail safely: every validation failure returns the user to a stable screen rather than terminating the application.
- Fail clearly: every error, warning, or confirmation uses a native message box so it is immediately visible and understandable.
- Fail early: input is validated immediately after it is entered, before any stock deduction or cart modification takes place, preventing inconsistent data.
- No silent failures: every rejected or cancelled action is explicitly reported to the user rather than being ignored.

11. Conclusion

This project is a complete, practical application of Object-Oriented Programming concepts, combining clear class design, well-separated responsibilities, a graphical user interface built with tkinter, and a simple sales/inventory statistics dashboard built with matplotlib. Thanks to its OOP-based structure, the system can be easily extended in the future — whether by adding new features, connecting it to a real database, or introducing customer accounts and multiple payment methods.